Week 6 Progress Report

Group G – University Student-Support Case Agent | Week 6 (5–9 Oct 2026): Memory, State and Interoperability

1. Summary

This week we made the workflow state explicit, added one justified persistent-memory use case with consent, retention and deletion controls, and documented and implemented the case-support capability as an MCP-style interface. All Week 1–5 behaviour is preserved: the 14 existing Week 5 tests pass unchanged alongside 32 new tests (46 total, all offline, no API key needed).

2. Deliverables

Deliverable	What was delivered	Location
State model	8-phase state machine with allowed-transition table, transition log and trace snapshot (raw student ID never logged)	src/state.py; docs/week6/Week 6 - Workflow State Model.docx
Memory Design and Data Handling Note	Use case, what is stored / not stored, access, consent, 90-day retention, deletion, risks	src/memory.py; docs/week6/Week 6 - Memory Design and Data Handling Note.docx
Memory/state demonstration	9-step offline demo covering consent, recall, duplicate warning, override, isolation, export, delete, expiry	src/demo_week6.py; docs/evidence/week6_demo_*
MCP-style interface spec	4 tools with scopes, schemas, error codes, 9 security controls, each mapped to a test	src/mcp_interface.py; docs/week6/Week 6 - MCP-Style Interface Specification.docx

3. Memory use case and how it stays under control

The agent remembers which follow-up tickets a student has already opened (ticket ID, case ID, category, case status). It does not store names or free-text descriptions. This lets it warn about a likely duplicate instead of silently creating a second ticket.

- Memory is advisory. A recent ticket (within 7 days) triggers a visible duplicate warning; the student can override it by asking for another ticket. Nothing is blocked or created silently.
- Memory never authorises anything: the Week 5 rules (explicit request, case checked, valid status, valid category) still apply.
- No consent means no reads and no writes. Revoking consent or calling forget() deletes the data; records expire after 90 days.
- If the memory store fails, the workflow continues exactly as in Week 5 and the failure is logged.

4. Evidence

- Test run: 46 tests, 0 failures (python -m unittest discover -s tests). New tests cover state transitions, data minimisation, student isolation, retention, deletion, audit log content, duplicate warning/override, memory outage, scope checks, schema validation and confirmation.
- Demo output saved to docs/evidence/week6_demo_console.txt and week6_demo_output.json.

5. Issues found and fixed

- The memory-outage test exposed a real bug: a failing memory read during case lookup would have crashed the run. It now degrades gracefully.

- Added a case-ownership check (a student cannot look up another student's case). A wrong owner returns the same message as 'not found', so case IDs cannot be probed.

6. Limitations (stated honestly)

- The demo and tests drive the application layer directly. They do not call the live Gemini/Groq model, so a live end-to-end run with student_id and memory is still to be captured by the team.
- The MCP-style interface is in-process JSON-RPC, not a networked MCP server and not the full official specification.
- Consent is granted in code for the demo; ownership data is synthetic; storage is local SQLite without encryption at rest; the 90-day retention is a project assumption.
- Duplicate-override detection is keyword-based, like the Week 5 ticket-request check.

7. Next steps (Week 7)

- Capture one live-model run using run_bounded_agent(message, student_id, memory) as additional evidence.
- Confirm retention period and consent wording with the course supervisor, then update the Data Handling Note.
- Update the architecture diagram to show the memory store and MCP-style interface.